

Guía n.º 1.3.4

Actividad: Herencia, clases abstractas, interfaces y polimorfismo en Java

Análisis de jerarquías de clases y reutilización de comportamiento en programación orientada a objetos

Guía n.º	1.3.4
Nombre de la guía	Actividad de análisis y aplicación de herencia, clases abstractas, interfaces y polimorfismo en Java
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Reconocer relaciones de herencia, comprender el uso de clases abstractas e interfaces y aplicar polimorfismo en ejemplos simples de Java.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Distingue clases base y derivadas, explica la utilidad de una clase abstracta, reconoce contratos definidos por interfaces y aplica polimorfismo en soluciones orientadas a objetos.

Propósito de la guía

Desarrollar la capacidad de modelar soluciones en Java usando mecanismos de reutilización y organización propios de la programación orientada a objetos. A través de un caso simple, el estudiante analizará cómo varias clases pueden compartir atributos y métodos mediante herencia, cómo una clase abstracta define una base común, cómo una interfaz establece comportamientos obligatorios y cómo el polimorfismo permite tratar objetos distintos de forma uniforme.

La guía presenta una jerarquía de vehículos como caso central. A partir de ella se explican los conceptos de herencia, clase abstracta, interfaz y polimorfismo, y luego se proponen ejercicios para aplicar lo aprendido en nuevos contextos.

Instrucciones generales

- Lee con atención el caso base antes de revisar los ejemplos en Java.
- Identifica qué elementos son comunes a todas las clases y cuáles pertenecen solo a una subclase.
- Observa cuándo se usa `extends`, cuándo se usa `implements` y por qué.
- Relaciona los métodos abstractos con la necesidad de obligar comportamientos en las clases hijas.
- Analiza cómo el polimorfismo permite trabajar con distintos objetos usando una misma referencia.
- Completa los ejercicios finales usando nombres claros y justificando tus decisiones de modelado.

Formato sugerido de respuesta para cada ejercicio

1. Nombre de la clase
2. Descripción breve de la responsabilidad de la clase
3. Lista de atributos
4. Lista de métodos
5. Constructor propuesto
6. Justificación breve: ¿qué datos y acciones fueron seleccionados y por qué?



Caso base: Sistema de vehículos

Situación: Una empresa necesita representar distintos tipos de vehículos dentro de un sistema. Todos los vehículos comparten datos básicos como marca, modelo y velocidad máxima, pero algunos tienen comportamientos particulares. Por ejemplo, un Auto puede abrir el maletero y una Moto puede hacer un caballito. Además, ciertos vehículos pueden tocar bocina, por lo que se decide definir un comportamiento común mediante una interfaz.

Consigna: Analiza las siguientes clases y observa cómo se aplican herencia, clase abstracta, interfaz y polimorfismo dentro de una misma solución en Java.

Clase abstracta base	<pre>public abstract class Vehiculo { protected String marca; protected String modelo; protected int velocidadMaxima; public Vehiculo(String marca, String modelo, int velocidadMaxima) { this.marca = marca; this.modelo = modelo; this.velocidadMaxima = velocidadMaxima; } public String getMarca() { return marca; } public String getModelo() { return modelo; } public int getVelocidadMaxima() { return velocidadMaxima; } public void mostrarDatos() { System.out.println("Marca: " + marca); System.out.println("Modelo: " + modelo); System.out.println("Velocidad máxima: " + velocidadMaxima); } public abstract void desplazarse(); }</pre>
Interfaz	<pre>public interface TocableBocina { void tocarBocina(); }</pre>

Subclase Auto	<pre>public class Auto extends Vehiculo implements TocableBocina { private int cantidadPuertas; public Auto(String marca, String modelo, int velocidadMaxima, int cantidadPuertas) {</pre>
----------------------	--

	<pre> super(marca, modelo, velocidadMaxima); this.cantidadPuertas = cantidadPuertas; } public int getCantidadPuertas() { return cantidadPuertas; } public void abrirMaletero() { System.out.println("El maletero del auto está abierto."); } @Override public void desplazarse() { System.out.println("El auto se desplaza por la carretera."); } @Override public void tocarBocina() { System.out.println("Bocina del auto: pi pi."); } } </pre>
Subclase Moto	<pre> public class Moto extends Vehiculo implements TocableBocina { private boolean tieneParabrisas; public Moto(String marca, String modelo, int velocidadMaxima, boolean tieneParabrisas) { super(marca, modelo, velocidadMaxima); this.tieneParabrisas = tieneParabrisas; } public boolean isTieneParabrisas() { return tieneParabrisas; } public void hacerCaballito() { System.out.println("La moto está haciendo un caballito."); } @Override public void desplazarse() { System.out.println("La moto se desplaza entre calles y avenidas."); } @Override public void tocarBocina() { System.out.println("Bocina de la moto: bip bip."); } } </pre>
Uso polimórfico	<pre> public class Main { public static void main(String[] args) { </pre>



	<pre>Vehiculo v1 = new Auto("Toyota", "Corolla", 180, 4); Vehiculo v2 = new Moto("Yamaha", "FZ", 140, true); v1.mostrarDatos(); v1.desplazarse(); v2.mostrarDatos(); v2.desplazarse(); } }</pre>
Descripción general	La solución usa una clase abstracta para reunir características comunes, una interfaz para definir un comportamiento obligatorio y dos subclases que implementan su propia versión del método desplazarse. Luego, mediante referencias de tipo Vehiculo, ambos objetos se usan de manera polimórfica.

Conceptos clave trabajados en la guía

Herencia	La herencia permite crear una clase nueva a partir de otra ya existente. La subclase reutiliza atributos y métodos de la clase padre y puede agregar o redefinir comportamientos.
Clase abstracta	Una clase abstracta es una clase que no se instancia directamente. Sirve como base para otras clases y puede contener métodos concretos y métodos abstractos.
Interfaz	Una interfaz define un contrato de comportamiento. Las clases que la implementan se comprometen a escribir los métodos declarados en ella.
Polimorfismo	El polimorfismo permite tratar objetos de distintas clases relacionadas como si fueran del mismo tipo base, ejecutando el comportamiento específico de cada uno.
Método sobrescrito	Un método sobrescrito es un método redefinido en una subclase para ofrecer una implementación propia del comportamiento heredado o abstracto.

¿Cómo se observa la herencia en este ejemplo?

La herencia aparece cuando Auto y Moto extienden la clase Vehiculo. Ambas clases comparten marca, modelo, velocidad máxima y el método mostrarDatos, evitando repetir código. Al mismo tiempo, cada una agrega atributos propios y comportamientos particulares, lo que demuestra cómo la herencia permite reutilizar y especializar una estructura común.

¿Cómo se observa el uso de una clase abstracta?

La clase Vehiculo se declara abstract porque representa una idea general que no se utilizará directamente como objeto independiente. Su función es servir de base para clases concretas como Auto y Moto. Además, incluye el método abstracto desplazarse(), obligando a cada subclase a definir cómo se mueve según su naturaleza.

¿Cómo se observa el uso de interfaces?



La interfaz Tocabocina declara el método tocarBocina() sin implementar su contenido. Tanto Auto como Moto implementan esa interfaz, pero cada una define su propia forma de tocar la bocina. Esto permite establecer comportamientos comunes sin depender de una sola jerarquía de herencia.

¿Cómo se observa el polimorfismo en este ejemplo?

El polimorfismo se aprecia cuando se crean referencias del tipo Vehiculo para guardar un Auto y una Moto. Aunque ambas variables tienen el mismo tipo base, al llamar al método desplazarse() se ejecuta la versión correspondiente a cada objeto real. Esto demuestra que diferentes clases pueden responder de manera distinta ante el mismo mensaje.

Relación entre los elementos del ejemplo

Elemento	Ejemplo en la guía	Función dentro del diseño
Clase abstracta	Vehiculo	Define la estructura común de todos los vehículos.
Subclase	Auto y Moto	Especializan el comportamiento y agregan características propias.
Interfaz	Tocabocina	Establece un comportamiento común que distintas clases deben implementar.
Método abstracto	desplazarse()	Obliga a cada subclase a definir cómo se desplaza.
Polimorfismo	Vehiculo v1 = new Auto(...);	Permite tratar objetos distintos como instancias del tipo base.

Ejemplo de uso ampliado

Código de ejemplo	<pre>public class PruebaVehiculos { public static void main(String[] args) { Vehiculo[] vehiculos = { new Auto("Toyota", "Yaris", 170, 4), new Moto("Honda", "CB190R", 150, false) }; for (Vehiculo vehiculo : vehiculos) { vehiculo.mostrarDatos(); vehiculo.desplazarse(); System.out.println("-----"); } } }</pre>
Explicación	Se crea un arreglo de tipo Vehiculo que almacena objetos de distintas subclases. En el recorrido, cada objeto responde con su propia versión del método desplazarse(), reforzando el uso del polimorfismo.

Actividades de aplicación

Ejercicio 1: Identificación de la jerarquía

Situación: Observa el caso base del sistema de vehículos.



Consigna: Identifica cuál es la clase abstracta, cuáles son las subclases y qué elementos comparten entre sí. Explica por qué Vehiculo no debería instanciarse directamente.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 2: Nueva subclase

Situación: La empresa ahora desea incorporar bicicletas al sistema.

Consigna: Propón una clase Bicicleta que herede de Vehiculo. Define al menos 4 atributos, 3 métodos y un constructor coherente con el contexto.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 3: Nueva interfaz

Situación: Además de tocar bocina, algunos vehículos pueden encender luces de emergencia.

Consigna: Diseña una interfaz nueva relacionada con ese comportamiento y explica qué clases del ejemplo podrían implementarla y por qué.



Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 4: Aplicación de polimorfismo

Situación: Se quiere recorrer una colección con distintos tipos de vehículos y ejecutar el mismo mensaje sobre todos ellos.

Consigna: Explica cómo aplicarías polimorfismo en este caso y escribe una propuesta simple de uso con objetos de distintas subclases.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 5: Adaptación del modelo

Situación: Un parque de diversiones necesita representar medios de transporte internos como trenes pequeños, carros eléctricos y bicicletas de alquiler.

Consigna: Crea una propuesta de jerarquía de clases basada en ese contexto. Define una clase abstracta base, al menos dos subclases y una interfaz que represente un comportamiento común.



Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
Comprensión de la herencia y la jerarquía	25%	Reconoce correctamente clases base, subclasses y elementos compartidos dentro del diseño.
Uso de clase abstracta e interfaces	25%	Distingue con claridad la función de una clase abstracta y de una interfaz en la solución propuesta.
Aplicación de polimorfismo	25%	Explica y utiliza adecuadamente referencias comunes para objetos de diferentes subclasses.
Claridad y justificación	25%	Presenta respuestas ordenadas, comprensibles y bien fundamentadas según el problema planteado.

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de reconocer cuándo conviene usar herencia para reutilizar código, cuándo una clase abstracta permite definir una base común, cuándo una interfaz ayuda a establecer comportamientos obligatorios y cómo el polimorfismo facilita trabajar con distintos objetos bajo una misma estructura general en Java.